

OXVERSE ACADEMY

Full Stack Development Curriculum

Professional Diploma · Cohort-based · Project-driven

DURATION

5 Months (20 Weeks)

TOTAL WEEKS

20

LEVEL

Beginner to Industry Ready

PROJECTS

20+

CAPSTONE

1 SaaS-Grade Full Stack Product

PROGRAMME GOAL

Train complete beginners into industry-ready full stack developers capable of independently designing, building, securing, testing, and deploying production-grade SaaS applications using modern JavaScript/TypeScript, React, Next.js on the frontend and Python (FastAPI and Django) on the backend.

OVERVIEW

A rigorous 20-week, project-driven full stack development program covering HTML5, modern CSS and Tailwind, JavaScript and TypeScript, React and Next.js on the frontend, and Python-based backend engineering with FastAPI and Django REST Framework, PostgreSQL, SQLAlchemy, Redis, Celery, authentication and authorization, payments with Stripe and Paystack, file uploads, real-time features, automated testing, Docker, CI/CD, cloud deployment, observability and system design — culminating in a fully architected, built, tested, and deployed SaaS capstone product and an interview-ready professional portfolio.

No 82, Century Bus Stop, Ago Palace Way, Okota, Lagos · +234 814 846 2776

Course Outline

1. **Week 1:** Web Foundations: HTML5 & Modern CSS
2. **Week 2:** Tailwind CSS & Modern UI Engineering
3. **Week 3:** JavaScript Fundamentals
4. **Week 4:** JavaScript in the Browser: DOM, Events & Async
5. **Week 5:** TypeScript for Application Development
6. **Week 6:** React Fundamentals
7. **Week 7:** React Deep Dive: Hooks, Context & Performance
8. **Week 8:** Next.js: The Production React Framework
9. **Week 9:** Python Fundamentals for Backend Engineers
10. **Week 10:** Backend APIs with FastAPI
11. **Week 11:** Databases: PostgreSQL & SQLAlchemy ORM
12. **Week 12:** Authentication, Authorization & Security
13. **Week 13:** Django & Django REST Framework
14. **Week 14:** Caching, Background Jobs & Real-Time Features
15. **Week 15:** Payments, File Uploads & Third-Party Integrations
16. **Week 16:** Testing, Docker & CI/CD
17. **Week 17:** Deployment, Observability & System Design
18. **Week 18:** Capstone Planning & SaaS Product Architecture
19. **Week 19:** Capstone Build Sprint
20. **Week 20:** Capstone Launch, Review & Career Readiness

Web Foundations: HTML5 & Modern CSS

Establish rock-solid fundamentals of semantic HTML5 and modern CSS layout systems used in production teams in 2026.

LEARNING OBJECTIVES

- Write semantic, accessible HTML5 documents
- Master Flexbox and CSS Grid for real layouts
- Understand the CSS box model, cascade and specificity
- Build responsive designs with mobile-first media queries

DETAILED TOPIC BREAKDOWN

1.1 HTML5 Document Structure

DOCTYPE and document skeleton Semantic tags: header, nav, main, section, article, aside, footer

Forms: input types, labels, fieldset, validation attributes Tables and when to use them

Accessibility basics: alt text, ARIA roles, landmark regions Meta tags for SEO and viewport Favicons and web manifest basics

Embedding media: picture, video, audio tags HTML boilerplate best practices

1.2 CSS Fundamentals

Selectors: type, class, id, attribute, pseudo-class, pseudo-element The box model: content, padding, border, margin

Cascade, specificity and inheritance rules Units: px, %, em, rem, vh, vw Color systems: hex, rgb, hsl, oklch

Typography: font-family stacks, line-height, letter-spacing CSS variables (custom properties)

Positioning: static, relative, absolute, fixed, sticky

1.3 Flexbox Layout

flex-direction, flex-wrap, flex-flow justify-content and align-items align-self and align-content

flex-grow, flex-shrink, flex-basis shorthand Building navbars with flexbox Building card rows with flexbox

Common flexbox layout bugs and fixes

1.4 CSS Grid Layout

grid-template-columns and grid-template-rows fr unit and repeat() function grid-gap / gap property

grid-template-areas for page layouts Auto-placement and grid-auto-flow minmax() and auto-fit/auto-fill for responsive grids

Combining Grid with Flexbox appropriately

1.5 Responsive Design

Mobile-first vs desktop-first strategy Media queries and breakpoints Fluid typography with clamp()

Responsive images with srcset and sizes Container queries (2026 browser support)

Testing responsiveness with devtools device toolbar

HANDS-ON EXERCISES

- Build a semantic personal profile page with forms
- Recreate a navbar using only Flexbox
- Build a 12-column responsive grid layout
- Convert a fixed-width page into a fluid responsive page
- Debug 5 broken CSS specificity examples

ASSIGNMENTS

- Build a fully responsive personal portfolio landing page using semantic HTML5 and CSS Grid/Flexbox
- Recreate a real-world website layout (e.g. a news homepage) pixel-close using only HTML/CSS

WEEKLY LEARNING OUTCOMES

- Confidently structure any webpage semantically
- Build responsive layouts without frameworks
- Debug CSS cascade and specificity issues
- Understand how browsers render the box model

WEEKLY ASSESSMENT

Graded review of the responsive portfolio project against a semantic HTML and layout accuracy checklist

PROJECTS

- Responsive Personal Portfolio Page

Tailwind CSS & Modern UI Engineering

Move from hand-written CSS to utility-first styling with Tailwind CSS, and learn component-driven UI thinking.

LEARNING OBJECTIVES

- Configure and use Tailwind CSS in a project
- Build fully responsive UIs with utility classes
- Create reusable design tokens and theme extensions
- Implement dark mode and animations with Tailwind

DETAILED TOPIC BREAKDOWN

2.1 Tailwind Setup & Core Concepts

Installing Tailwind via Vite/PostCSS tailwind.config.js structure Utility-first philosophy vs traditional CSS

Spacing scale, sizing scale, color palette Responsive prefixes: sm, md, lg, xl, 2xl State variants: hover, focus, active, disabled

Group and peer modifiers

2.2 Layout Utilities

Flex and grid utility classes Container and max-width utilities Position and z-index utilities Gap, space-x/space-y utilities

Aspect-ratio utilities Overflow and truncation utilities

2.3 Design Systems with Tailwind

Extending theme colors, fonts, spacing Creating reusable component classes with @apply Typography plugin for prose content

Building a button/badge/card component library Consistent shadow and radius tokens

2.4 Dark Mode & Animation

Class-based vs media-based dark mode Toggling dark mode with JS and localStorage Transition and animation utilities

Keyframe animations in Tailwind config Using tailwindcss-animate plugin

2.5 Component Libraries & Tooling

Introduction to shadcn/ui and Radix primitives Using clsx/cn utility for conditional classes Linting with Prettier + Tailwind plugin

Organizing large Tailwind projects

HANDS-ON EXERCISES

- Rebuild Week 1's portfolio using Tailwind only
- Build a pricing card component with hover states
- Implement dark mode toggle with persistence
- Create a reusable Button component with variants

ASSIGNMENTS

- Design and build a marketing landing page (hero, features, pricing, footer) fully responsive with Tailwind
- Build a small component library (buttons, cards, badges, alerts) with dark mode support

PROJECTS

- SaaS Marketing Landing Page with Dark Mode

WEEKLY LEARNING OUTCOMES

- Build production UIs quickly with utility-first CSS
- Create and maintain a design token system
- Ship accessible, themeable dark mode
- Understand shadcn/ui component conventions

WEEKLY ASSESSMENT

Peer + instructor review of the landing page against responsiveness, dark mode and design consistency criteria

JavaScript Fundamentals

Deep dive into core JavaScript: syntax, data structures, functions, and the mental models needed before frameworks.

LEARNING OBJECTIVES

- Write clean modern ES2024+ JavaScript
- Master arrays, objects and their built-in methods
- Understand scope, closures, and hoisting
- Work confidently with functions and higher-order functions

DETAILED TOPIC BREAKDOWN

3.1 Syntax & Data Types

let, const, var differences Primitive types vs reference types Template literals and string methods
Truthy/falsy values and type coercion Operators: nullish coalescing, optional chaining Type checking with typeof and instanceof

3.2 Control Flow & Functions

if/else, switch, ternary expressions for, for...of, for...in, while loops Function declarations vs expressions vs arrow functions
Default parameters and rest parameters Closures and lexical scope IIFEs and their use cases

3.3 Arrays & Objects

Array methods: map, filter, reduce, find, some, every Array destructuring and spread operator
Object methods: keys, values, entries, assign Object destructuring with defaults and renaming
Immutability patterns for arrays/objects JSON.stringify and JSON.parse

3.4 Advanced Functions & Scope

Higher-order functions Currying and function composition this keyword in different contexts call, apply, bind
Hoisting behavior of var/let/const/functions

3.5 Error Handling & Modules

try/catch/finally Custom Error classes ES Modules: import/export syntax CommonJS vs ESM differences
Debugging with console methods and browser devtools

HANDS-ON EXERCISES

- Implement your own map, filter, reduce from scratch
- Solve 15 array/object manipulation katas
- Build a closures-based counter and memoizer
- Refactor callback code into cleaner functional style

ASSIGNMENTS

- Build a command-line-style budget tracker using arrays/objects and functions
- Build a set of 10 utility functions (debounce, throttle, deepClone, groupBy, etc.) with tests

PROJECTS

- JavaScript Utility Library

WEEKLY LEARNING OUTCOMES

- Think in JavaScript idioms fluently
- Manipulate complex data structures confidently
- Understand closures, scope and this deeply
- Debug JS programs using devtools

WEEKLY ASSESSMENT

Timed coding assessment covering arrays, objects, closures and functions

JavaScript in the Browser: DOM, Events & Async

Learn to manipulate the DOM, handle events, and master asynchronous JavaScript including Promises and async/await.

LEARNING OBJECTIVES

- Manipulate the DOM programmatically
- Handle user events and event delegation
- Fetch data from APIs using fetch and async/await
- Understand the JavaScript event loop

DETAILED TOPIC BREAKDOWN

4.1 DOM Manipulation

querySelector/querySelectorAll createElement, appendChild, remove classList API Modifying attributes and styles via JS
Traversing the DOM tree textContent vs innerHTML vs innerText security implications

4.2 Events

addEventListener and event object Event bubbling and capturing Event delegation pattern Form events and preventDefault
Keyboard and mouse events Debouncing/throttling event handlers

4.3 Asynchronous JavaScript

Callbacks and callback hell Promises: then, catch, finally Promise.all, Promise.race, Promise.allSettled
async/await syntax and error handling Microtasks vs macrotasks and the event loop
setTimeout, setInterval, requestAnimationFrame

4.4 Working with APIs

fetch API and Response object GET/POST/PUT/DELETE requests with fetch Handling JSON responses and errors
CORS explained AbortController for cancelling requests Building a simple API client wrapper

4.5 Browser Storage & Tooling

localStorage and sessionStorage Cookies basics Browser devtools: Network, Console, Sources panels
Introduction to Vite as a dev server/bundler

HANDS-ON EXERCISES

- Build a dynamic to-do list with full CRUD in vanilla JS
- Implement event delegation for a large list
- Consume a public REST API and render results
- Build a debounced live search box

ASSIGNMENTS

- Build a weather dashboard app that fetches live data from a public API
- Build a client-side quiz app with timer, scoring and localStorage persistence

PROJECTS

- Vanilla JS Weather Dashboard

WEEKLY LEARNING OUTCOMES

- Manipulate and update the DOM efficiently
- Handle complex event-driven interactions
- Fetch and render async data cleanly
- Understand the JS event loop deeply

WEEKLY ASSESSMENT

Practical project demo plus a written quiz on the event loop and async patterns

TypeScript for Application Development

Add static typing to JavaScript knowledge; learn TypeScript deeply as the foundation for React and Next.js work ahead.

LEARNING OBJECTIVES

- Configure TypeScript in a project
- Use interfaces, types and generics effectively
- Understand TS utility types
- Migrate JavaScript code to strict TypeScript

DETAILED TOPIC BREAKDOWN

5.1 TypeScript Basics

tsconfig.json key options Primitive types, arrays, tuples Type inference vs explicit typing
any, unknown, never, void differences Type aliases vs interfaces

5.2 Functions & Objects in TS

Typing function parameters and return values Optional and default parameters Object types and readonly properties
Union and intersection types Discriminated unions

5.3 Generics

Generic functions Generic interfaces and types Constraints with extends Generic utility patterns for reusable code

5.4 Advanced Types

Utility types: Partial, Pick, Omit, Record, Required Mapped types Conditional types basics Type guards and narrowing
satisfies operator

5.5 Tooling & Practice

Strict mode configuration ESLint + TypeScript integration Typing third-party libraries with @types
Migrating a JS project to TS incrementally

HANDS-ON EXERCISES

- Convert the Week 3 utility library to strict TypeScript
- Write 10 generic function exercises
- Model a domain (e.g. e-commerce) with interfaces and unions
- Fix type errors in a deliberately broken TS file

ASSIGNMENTS

- Migrate the vanilla JS weather dashboard to TypeScript with full type safety
- Build a typed data-modeling exercise for a blog platform (Post, Author, Comment types)

PROJECTS

- TypeScript Migration Project

WEEKLY LEARNING OUTCOMES

- Write type-safe, self-documenting JavaScript
- Use generics to build reusable, safe abstractions
- Read and resolve complex TS compiler errors
- Be fully prepared to learn React with TypeScript

WEEKLY ASSESSMENT

Code review of the migrated TypeScript project for type safety and correctness

React Fundamentals

Learn React from first principles: components, JSX, props, state, and the component lifecycle using functional components and hooks.

LEARNING OBJECTIVES

- Build component-based UIs with JSX
- Manage local state with useState
- Understand props and component composition
- Handle events and conditional rendering in React

DETAILED TOPIC BREAKDOWN

6.1 React & JSX Basics

What React is and the virtual DOM concept JSX syntax rules and expressions Rendering lists with map and the key prop Conditional rendering patterns Fragments

6.2 Components & Props

Function components Props and prop typing with TypeScript Children prop and composition Default props patterns Component file/folder organization

6.3 State & Events

useState hook fundamentals Handling form inputs (controlled components) Event handlers in JSX Lifting state up Derived state vs redundant state

6.4 Component Lifecycle with Hooks

useEffect for side effects Dependency arrays and cleanup functions Fetching data with useEffect Common useEffect pitfalls

6.5 Project Setup & Tooling

Creating a React app with Vite + TypeScript Folder structure conventions React DevTools ESLint/Prettier setup for React projects

HANDS-ON EXERCISES

- Build a counter and toggle component with useState
- Build a controlled multi-field form with validation feedback
- Render a filterable, sortable list of items
- Fetch and display API data with loading/error states

ASSIGNMENTS

- Build a task manager app (add/edit/delete/filter tasks) using only local state
- Build a movie search app consuming a public API with loading and error UI states

PROJECTS

- React Task Manager App

WEEKLY LEARNING OUTCOMES

- Build interactive UIs using React components
- Manage component state and side effects correctly
- Structure a React + TypeScript project professionally
- Debug React apps using DevTools

WEEKLY ASSESSMENT

Live coding assessment: build a small feature component under time constraints

React Deep Dive: Hooks, Context & Performance

Master advanced React patterns including custom hooks, Context API, performance optimization, and routing with React Router.

LEARNING OBJECTIVES

- Build and use custom hooks
- Manage global state with Context API
- Optimize React performance with memoization
- Implement client-side routing

DETAILED TOPIC BREAKDOWN

7.1 Advanced Hooks

useRef for DOM access and mutable values useMemo for expensive computations useCallback for stable function references
useReducer for complex state logic Rules of hooks explained

7.2 Custom Hooks

Extracting reusable logic into custom hooks useFetch, useDebounce, useLocalStorage patterns Composing multiple hooks
Testing custom hooks

7.3 Context API & State Sharing

createContext and useContext Provider pattern Avoiding unnecessary re-renders with context
When to use Context vs prop drilling vs external state libraries

7.4 Performance Optimization

React.memo for component memoization Identifying re-render issues with React DevTools Profiler
Code-splitting with React.lazy and Suspense Virtualization concepts for long lists

7.5 Routing with React Router

BrowserRouter, Routes, Route setup Nested routes and layouts useNavigate, useParams, useSearchParams
Protected routes pattern Route-based code splitting

HANDS-ON EXERCISES

- Build 3 custom hooks (useToggle, useFetch, useDebounce)
- Refactor a prop-drilled app to use Context
- Profile and fix an intentionally slow React component
- Build a multi-page app with nested routing

ASSIGNMENTS

- Build a multi-page e-commerce product catalog with cart state via Context and routing
- Optimize a given slow React app and document performance improvements with before/after profiler screenshots

PROJECTS

- Multi-Page Product Catalog with Cart

WEEKLY LEARNING OUTCOMES

- Write reusable custom hooks confidently
- Share state across a React app idiomatically
- Diagnose and fix React performance issues
- Implement production-grade client-side routing

WEEKLY ASSESSMENT

Performance audit assignment graded on measurable improvement plus routing implementation review

Next.js: The Production React Framework

Transition from client-only React to Next.js (App Router), covering routing, rendering strategies, and data fetching for production apps.

LEARNING OBJECTIVES

- Build applications using the Next.js App Router
- Understand server components vs client components
- Implement SSR, SSG and ISR rendering strategies
- Handle data fetching, loading and error states in Next.js

DETAILED TOPIC BREAKDOWN

8.1 Next.js App Router Basics

Project structure with the app directory File-based routing conventions layout.tsx, page.tsx, loading.tsx, error.tsx
Nested layouts and route groups Dynamic routes and catch-all segments

8.2 Server & Client Components

Server Components by default explained 'use client' directive and when to use it
Passing data between server and client components Composition patterns to minimize client bundle size

8.3 Data Fetching & Rendering Strategies

fetch() caching behavior in Next.js Static Site Generation (SSG) with generateStaticParams
Server-Side Rendering (SSR) on demand Incremental Static Regeneration (ISR) with revalidate Streaming with Suspense

8.4 Server Actions & Forms

Defining Server Actions Form submission with Server Actions Optimistic UI updates with useOptimistic
Revalidating data with revalidatePath/revalidateTag

8.5 Metadata, Images & Assets

generateMetadata for SEO next/image optimization next/font for font optimization
Static assets and public folder conventions

HANDS-ON EXERCISES

- Convert a client-rendered React page to Next.js App Router
- Build a blog with SSG using generateStaticParams
- Implement a Server Action for a contact form
- Add SEO metadata to all pages of a sample site

ASSIGNMENTS

- Build a fully server-rendered blog platform with dynamic routes, ISR, and optimized images/fonts
- Rebuild the Week 7 product catalog in Next.js with proper server/client component boundaries

PROJECTS

- Server-Rendered Blog Platform with ISR

WEEKLY LEARNING OUTCOMES

- Architect apps correctly between server and client components
- Choose the right rendering strategy per page
- Use Server Actions for mutations without a separate API layer
- Optimize Next.js apps for SEO and performance

WEEKLY ASSESSMENT

Project review verifying correct rendering strategy choices and server/client component boundaries

Python Fundamentals for Backend Engineers

Build strong Python fundamentals with a backend-engineering lens, preparing for FastAPI and Django in coming weeks.

LEARNING OBJECTIVES

- Write clean, idiomatic Python 3.12+ code
- Use Python's data structures and comprehensions effectively
- Understand object-oriented programming in Python
- Work with virtual environments and package management

DETAILED TOPIC BREAKDOWN

9.1 Python Syntax & Data Types

Variables, dynamic typing, type hints Numbers, strings, booleans, None Lists, tuples, sets, dictionaries
String formatting with f-strings Slicing and indexing

9.2 Control Flow & Functions

if/elif/else, match-case statements for/while loops, range, enumerate, zip Function definitions, default and keyword args
*args and **kwargs Lambda functions List/dict/set comprehensions

9.3 Object-Oriented Python

Classes, __init__, instance vs class attributes Inheritance and method overriding Dunder methods: __str__, __repr__, __eq__
Properties with @property Dataclasses for data containers Abstract base classes basics

9.4 Modules, Packages & Environments

Importing modules and packages Creating virtual environments with venv/uv Installing packages with pip/uv
requirements.txt and pyproject.toml Organizing a Python project structure

9.5 Error Handling, Files & Type Hints

try/except/else/finally Custom exceptions Reading/writing files and context managers (with)
Type hints: List, Dict, Optional, Union Static type checking with mypy basics

HANDS-ON EXERCISES

- Solve 15 Python data structure and comprehension exercises
- Build a class hierarchy modeling a library system
- Write a CLI script that reads/writes JSON files
- Set up a Python project with venv and pyproject.toml

ASSIGNMENTS

- Build a command-line inventory management tool using OOP and file persistence
- Write a typed Python module with full mypy compliance and unit tests using pytest

PROJECTS

- Python CLI Inventory Manager

WEEKLY LEARNING OUTCOMES

- Write idiomatic, type-hinted Python code
- Apply OOP principles in real programs
- Manage Python environments and dependencies properly
- Be fully prepared for FastAPI/Django backend work

WEEKLY ASSESSMENT

Timed Python coding assessment plus code review of the CLI project

Backend APIs with FastAPI

Build robust REST APIs using FastAPI, covering routing, validation with Pydantic, dependency injection, and async request handling.

LEARNING OBJECTIVES

- Build REST APIs with FastAPI routers
- Validate request/response data with Pydantic models
- Use FastAPI's dependency injection system
- Understand async request handling in Python

DETAILED TOPIC BREAKDOWN

10.1 FastAPI Fundamentals

Installing and running FastAPI with uvicorn Path operations: GET, POST, PUT, PATCH, DELETE Path and query parameters
Automatic OpenAPI/Swagger docs Project structure for scalable FastAPI apps

10.2 Pydantic Models & Validation

Defining Pydantic v2 models Field validation and constraints Nested models and lists of models
Request body vs response_model Custom validators

10.3 Dependency Injection

Depends() fundamentals Reusable dependencies for pagination, filtering Dependency chains and sub-dependencies
Class-based dependencies

10.4 Routers & Project Structure

APIRouter for modular routes Organizing routers, schemas, services, models in folders Middleware basics (CORS, logging)
Exception handlers and custom HTTPException

10.5 Async Python & Concurrency

async def vs def endpoints When FastAPI runs code in a threadpool asyncio basics: await, coroutines
Async HTTP calls with httpx

HANDS-ON EXERCISES

- Build a CRUD API for a 'notes' resource with Pydantic validation
- Add pagination and filtering dependencies to a list endpoint
- Write custom exception handlers for common error cases
- Call an external API asynchronously from a FastAPI endpoint

ASSIGNMENTS

- Build a fully documented Task Management REST API with FastAPI (in-memory store)
- Refactor the API into routers/schemas/services layers following clean architecture

PROJECTS

- Task Management REST API with FastAPI

WEEKLY LEARNING OUTCOMES

- Design and build REST APIs following FastAPI best practices
- Validate and shape data reliably with Pydantic
- Structure backend codebases for scalability
- Understand when and how to use async in Python APIs

WEEKLY ASSESSMENT

API design review using the auto-generated Swagger docs and a Postman/Insomnia test collection

Databases: PostgreSQL & SQLAlchemy ORM

Learn relational database design and connect FastAPI applications to PostgreSQL using SQLAlchemy 2.0 and Alembic migrations.

LEARNING OBJECTIVES

- Design normalized relational database schemas
- Write SQL queries confidently
- Use SQLAlchemy ORM models and relationships
- Manage schema changes with Alembic migrations

DETAILED TOPIC BREAKDOWN

11.1 Relational Database Design

Tables, rows, columns, primary/foreign keys Normalization (1NF, 2NF, 3NF) One-to-many, many-to-many relationships
Indexes and when to use them ER diagrams for schema planning

11.2 SQL Fundamentals

SELECT, WHERE, ORDER BY, LIMIT JOINS: INNER, LEFT, RIGHT, FULL Aggregate functions and GROUP BY/HAVING
Subqueries and CTEs INSERT, UPDATE, DELETE, transactions

11.3 SQLAlchemy ORM (2.0 style)

Engine and session configuration Declarative models and mapped_column Relationships: relationship(), back_populates
Querying with select() statement style Eager vs lazy loading strategies

11.4 Integrating SQLAlchemy with FastAPI

Dependency-injected DB sessions CRUD service layer patterns Connecting Pydantic schemas with ORM models
Pagination and filtering at the DB level

11.5 Migrations & Data Integrity

Alembic setup and configuration Generating and applying migrations Rolling back migrations safely
Database constraints: unique, check, not null Seeding data for development

HANDS-ON EXERCISES

- Design an ER diagram for a multi-tenant SaaS schema
- Write 10 SQL queries involving joins and aggregates
- Build SQLAlchemy models with relationships for blog posts/comments/authors
- Create and apply an Alembic migration for a schema change

ASSIGNMENTS

- Rebuild the Task Management API with PostgreSQL persistence via SQLAlchemy and Alembic migrations
- Design and implement a normalized schema for a multi-vendor marketplace

WEEKLY LEARNING OUTCOMES

- Design correct, normalized relational schemas
- Write and optimize SQL queries
- Use SQLAlchemy ORM productively with FastAPI
- Manage database schema evolution safely with Alembic

WEEKLY ASSESSMENT

Database design review plus a live SQL query assessment

PROJECTS

- Persistent Task Management API with PostgreSQL

Authentication, Authorization & Security

Implement secure authentication and authorization flows, and learn essential web application security practices.

LEARNING OBJECTIVES

- Implement JWT-based authentication in FastAPI
- Implement role-based access control (RBAC)
- Apply password hashing and secure session handling
- Identify and mitigate common web vulnerabilities

DETAILED TOPIC BREAKDOWN

12.1 Authentication Fundamentals

Password hashing with bcrypt/argon2 JWT structure: header, payload, signature Access tokens vs refresh tokens

OAuth2 password flow in FastAPI (OAuth2PasswordBearer)

Storing tokens securely on the client (httpOnly cookies vs localStorage tradeoffs)

12.2 Implementing Auth in FastAPI

User registration and login endpoints Issuing and verifying JWTs Dependency-based route protection

Refresh token rotation strategy Email verification flow basics

12.3 Authorization & RBAC

Role and permission models in the database Role-based route guards Resource-based (ownership) authorization checks

Admin vs user vs org-level permissions for SaaS

12.4 OAuth & Third-Party Auth

OAuth2 authorization code flow concept Social login with Google/GitHub (conceptual + implementation)

Using Authlib or similar libraries in FastAPI

12.5 Web Security Essentials

OWASP Top 10 overview SQL injection prevention with ORM parameterization XSS and CSRF prevention strategies

Rate limiting and brute-force protection Secrets management with environment variables HTTPS, CORS configuration hardening

HANDS-ON EXERCISES

- Implement register/login/refresh endpoints with JWT
- Add role-based route protection to an existing API
- Implement Google OAuth login on a sample app
- Write a security checklist audit for a given codebase

ASSIGNMENTS

- Add full authentication and RBAC (admin/user roles) to the Task Management API
- Implement Google OAuth social login alongside email/password auth

PROJECTS

- Secure Auth System with JWT, RBAC and OAuth

WEEKLY LEARNING OUTCOMES

- Implement secure, production-grade authentication flows
- Enforce authorization rules correctly across an API
- Recognize and prevent common web vulnerabilities
- Manage secrets and security configuration properly

WEEKLY ASSESSMENT

Security-focused code review and a mock penetration test checklist walkthrough

Django & Django REST Framework

Learn Django as a batteries-included alternative/complement to FastAPI, including the ORM, admin panel, and building REST APIs with DRF.

LEARNING OBJECTIVES

- Build a Django project with apps and models
- Use Django ORM for data modeling and queries
- Build REST APIs with Django REST Framework
- Leverage Django's admin panel for internal tooling

DETAILED TOPIC BREAKDOWN

13.1 Django Fundamentals

Django project vs app structure settings.py configuration URL routing with urls.py Django's MTV architecture
manage.py commands

13.2 Django ORM

Defining models and fields Migrations: makemigrations, migrate QuerySets: filter, exclude, annotate, aggregate
Model relationships: ForeignKey, ManyToMany, OneToOne Model Meta options and custom managers

13.3 Django Admin & Views

Registering models in the admin site Customizing ModelAdmin Class-based views vs function-based views
Django templates basics (for admin tooling/dashboards)

13.4 Django REST Framework

Serializers and ModelSerializer APIView, generic views, and ViewSets Routers for automatic URL generation
Permissions and authentication classes in DRF

13.5 DRF Advanced Features

Pagination, filtering, and search in DRF Nested serializers for related data Throttling and rate limiting in DRF
Choosing Django+DRF vs FastAPI per use case

HANDS-ON EXERCISES

- Build a Django app with 3 related models and admin registration
- Build a DRF ViewSet with full CRUD and permissions
- Add filtering/search/pagination to a DRF endpoint
- Compare a feature built in both FastAPI and DRF

ASSIGNMENTS

- Build a Content Management API (articles, categories, authors) using Django + DRF with a working admin panel
- Add role-based permissions and nested serializers to the CMS API

PROJECTS

- Content Management API with Django REST Framework

WEEKLY LEARNING OUTCOMES

- Build production APIs with Django REST Framework
- Use Django's admin panel for rapid internal tooling
- Compare and choose between FastAPI and Django appropriately
- Model complex relational data with the Django ORM

WEEKLY ASSESSMENT

Project demo comparing DRF and FastAPI implementations of the same feature

Caching, Background Jobs & Real-Time Features

Add production capabilities: Redis caching, Celery background jobs/scheduling, and real-time features with WebSockets.

LEARNING OBJECTIVES

- Use Redis for caching and rate limiting
- Implement background job processing with Celery
- Build real-time features using WebSockets
- Design scheduled/periodic tasks

DETAILED TOPIC BREAKDOWN

14.1 Redis Fundamentals

Redis data types: strings, hashes, lists, sets, sorted sets Setting up Redis locally and in Docker
Connecting to Redis from Python with redis-py Using Redis for caching expensive queries Cache invalidation strategies

14.2 Rate Limiting & Sessions

Token bucket / sliding window rate limiting with Redis Using Redis for session storage Distributed locks with Redis

14.3 Background Jobs with Celery

Celery architecture: broker, worker, backend Configuring Celery with Redis/RabbitMQ as broker
Defining and calling tasks with `.delay()`/`.apply_async()` Retries, timeouts, and error handling in tasks
Celery Beat for scheduled/periodic tasks

14.4 Real-Time with WebSockets

WebSocket protocol basics Implementing WebSocket endpoints in FastAPI Django Channels overview for real-time in Django
Building a live notification/chat feature Broadcasting events with Redis Pub/Sub

14.5 Email & Notifications

Sending transactional email (SMTP/SendGrid/Resend/Postmark) Email templates for verification/password reset
Queuing email sending via Celery In-app notification patterns

HANDS-ON EXERCISES

- Cache a slow DB query with Redis and measure the improvement
- Build a Celery task that sends a welcome email asynchronously
- Implement a scheduled daily report task with Celery Beat
- Build a simple WebSocket-based live counter or chat

ASSIGNMENTS

- Add Redis caching, async email sending via Celery, and a real-time notification system to the Task Management API
- Implement a live chat feature using WebSockets and Redis Pub/Sub broadcasting

WEEKLY LEARNING OUTCOMES

- Use Redis effectively for caching and rate limiting
- Offload slow work to background jobs reliably
- Build real-time features with WebSockets
- Send transactional emails asynchronously

WEEKLY ASSESSMENT

Load-test demonstration showing performance improvement from caching and background job offloading

PROJECTS

- Real-Time Notifications & Background Jobs System

Payments, File Uploads & Third-Party Integrations

Integrate payment processing (Stripe/Paystack), handle file uploads to cloud storage, and consume third-party APIs professionally.

LEARNING OBJECTIVES

- Integrate Stripe and Paystack payment flows
- Handle file/image uploads to cloud storage
- Design and consume webhooks securely
- Build resilient third-party API integrations

DETAILED TOPIC BREAKDOWN

15.1 Payment Integration: Stripe

- Stripe Checkout vs Payment Intents
- Creating products/prices and checkout sessions
- Handling successful/failed payment redirects
- Stripe webhooks for payment confirmation
- Subscription billing basics with Stripe

15.2 Payment Integration: Paystack

- Paystack transaction initialization
- Verifying transactions server-side
- Paystack webhooks and signature verification
- Handling local currency and split payments

15.3 Webhooks & Idempotency

- Verifying webhook signatures
- Idempotency keys to prevent duplicate processing
- Retrying and logging webhook events
- Testing webhooks locally with tunneling tools

15.4 File Uploads & Storage

- Handling multipart/form-data uploads in FastAPI/Django
- Validating file type and size
- Uploading files to S3-compatible storage (S3/Cloudflare R2/Supabase Storage)
- Generating signed/pre-signed URLs
- Image processing/resizing basics

15.5 Third-Party API Integration Patterns

- Designing a resilient API client wrapper
- Retry with exponential backoff
- Timeouts and circuit breaker concepts
- API key management and rotation

HANDS-ON EXERCISES

- Integrate Stripe Checkout for a one-time payment flow
- Integrate Paystack for a subscription payment flow
- Build a secure file upload endpoint storing to S3-compatible storage
- Verify and process a webhook end-to-end

ASSIGNMENTS

- Add a subscription billing system (Stripe or Pystack) with webhook-based plan activation to the SaaS project
- Build a profile/avatar and document upload feature with cloud storage and image resizing

PROJECTS

- Subscription Billing & File Upload System

WEEKLY LEARNING OUTCOMES

- Integrate real payment providers confidently
- Handle webhooks securely and idempotently
- Manage file uploads to cloud storage safely
- Build resilient integrations with third-party APIs

WEEKLY ASSESSMENT

End-to-end demo of a successful test payment flow with webhook-confirmed activation

Testing, Docker & CI/CD

Build a professional engineering workflow: automated testing, containerization with Docker, and continuous integration/deployment pipelines.

LEARNING OBJECTIVES

- Write unit, integration and API tests for Python and React code
- Containerize full-stack applications with Docker
- Set up CI/CD pipelines with GitHub Actions
- Apply Git workflows used in professional teams

DETAILED TOPIC BREAKDOWN

16.1 Backend Testing

pytest fundamentals: fixtures, parametrize Testing FastAPI endpoints with TestClient/httpx
Testing Django/DRF views with APIClient Mocking external services and DB in tests Test database setup and teardown strategies
Code coverage with coverage.py

16.2 Frontend Testing

Unit testing with Vitest/Jest Testing React components with React Testing Library Mocking API calls in frontend tests
End-to-end testing with Playwright/Cypress

16.3 Docker Fundamentals

Images vs containers Writing a Dockerfile for a Python backend Writing a Dockerfile for a Next.js frontend
Multi-stage builds for smaller images docker-compose for multi-service local dev (app, Postgres, Redis)

16.4 Git & Team Workflows

Git branching strategies (trunk-based, GitFlow) Pull requests and code review etiquette
Conventional commits and semantic versioning Resolving merge conflicts confidently

16.5 CI/CD with GitHub Actions

Writing GitHub Actions workflows Running tests and linters automatically on PRs Building and pushing Docker images in CI
Deploying automatically on merge to main Environment secrets management in CI

HANDS-ON EXERCISES

- Write pytest tests achieving 80%+ coverage for one API module
- Write React Testing Library tests for a form component
- Dockerize the FastAPI backend and Postgres with docker-compose
- Write a GitHub Actions workflow that runs tests on every PR

ASSIGNMENTS

- Set up a full CI/CD pipeline that tests, builds, and deploys the SaaS project's backend and frontend on every merge
- Achieve and document meaningful test coverage across backend and critical frontend components

WEEKLY LEARNING OUTCOMES

- Write meaningful automated tests across the stack
- Containerize applications for consistent environments
- Build and maintain CI/CD pipelines
- Collaborate using professional Git workflows

WEEKLY ASSESSMENT

Review of CI pipeline run history and test coverage report

PROJECTS

- Fully Tested & Dockerized SaaS App with CI/CD Pipeline

Deployment, Observability & System Design

Deploy full-stack applications to production infrastructure, add observability, and learn system design fundamentals for scaling SaaS products.

LEARNING OBJECTIVES

- Deploy Next.js and Python backends to production platforms
- Configure custom domains, SSL and environment variables
- Implement logging, monitoring and error tracking
- Apply core system design principles for scalable apps

DETAILED TOPIC BREAKDOWN

17.1 Frontend Deployment

Deploying Next.js to Vercel Environment variables and build configuration Custom domains and SSL
Preview deployments for PRs Edge/CDN caching concepts

17.2 Backend Deployment

Deploying FastAPI/Django with Docker to Railway/Render/Fly.io Managed PostgreSQL and Redis in production
Zero-downtime deployment strategies Health checks and readiness probes Horizontal scaling basics

17.3 Observability

Structured logging best practices Error tracking with Sentry Application performance monitoring (APM) basics
Uptime monitoring and alerting Dashboards with Grafana/metrics basics

17.4 System Design Fundamentals

Load balancers and reverse proxies Vertical vs horizontal scaling Caching layers in system design
Database read replicas and sharding basics CAP theorem overview

17.5 Designing a SaaS Architecture

Multi-tenancy patterns (shared DB vs schema vs DB per tenant) API rate limiting at scale Designing for high availability
Cost-aware infrastructure decisions for early-stage SaaS

HANDS-ON EXERCISES

- Deploy a Next.js app to Vercel with a custom domain
- Deploy a Dockerized FastAPI app to Render/Railway/Fly.io
- Set up Sentry error tracking on both frontend and backend
- Draw a system design diagram for a given scaling scenario

ASSIGNMENTS

- Deploy the full SaaS project (frontend + backend + DB + Redis) to production with monitoring and error tracking
- Produce a system design document for scaling the SaaS project to 100k users

WEEKLY LEARNING OUTCOMES

- Deploy full-stack apps to real production infrastructure
- Set up meaningful observability for production systems
- Reason about system design tradeoffs confidently
- Plan infrastructure decisions appropriate to SaaS stage

WEEKLY ASSESSMENT

Live production deployment review plus system design document evaluation

PROJECTS

- Production Deployment of Full SaaS Stack with Monitoring

Capstone Planning & SaaS Product Architecture

Plan and architect the capstone SaaS product: requirements, data modeling, API design, and UI/UX before building.

LEARNING OBJECTIVES

- Define a scoped, buildable SaaS product idea
- Design a complete database schema and API contract
- Plan authentication, billing and core feature architecture
- Create a professional project plan and timeline

DETAILED TOPIC BREAKDOWN

18.1 Product Scoping

- Identifying a real problem and target user
- Defining MVP feature scope vs future roadmap
- Writing user stories and acceptance criteria
- Competitive analysis of similar SaaS products

18.2 Technical Architecture Planning

- Choosing the tech stack per feature (FastAPI vs Django, Next.js rendering strategy)
- Designing the full database schema with ER diagrams
- Designing the REST API contract (endpoints, request/response shapes)
- Planning multi-tenancy and role structure

18.3 UI/UX Planning

- Wireframing key screens (dashboard, billing, settings)
- Designing the information architecture/navigation
- Establishing a design system (colors, typography, components) in Tailwind
- Accessibility considerations for the product

18.4 Feature & Integration Planning

- Planning payment/subscription tiers and Stripe/Paystack integration
- Planning file upload and email touchpoints
- Planning background jobs and real-time features needed
- Planning testing and deployment strategy for the build phase

18.5 Project Management

- Breaking the capstone into weekly milestones
- Setting up a project board (GitHub Projects/Linear)
- Writing a technical README and architecture doc
- Risk assessment and scope-cutting strategy

HANDS-ON EXERCISES

- Write 15+ user stories with acceptance criteria for the capstone idea
- Produce a full ER diagram and API contract document
- Wireframe 5 key screens of the SaaS product
- Create a 2-week milestone plan with a project board

ASSIGNMENTS

- Produce a complete Product Requirements Document (PRD) and technical architecture document for the capstone
- Get instructor sign-off on scope, schema, API contract and wireframes before build begins

PROJECTS

- Capstone SaaS Product Requirements & Architecture Document

WEEKLY LEARNING OUTCOMES

- Scope a SaaS product realistically for a solo/small team build
- Design a coherent schema and API contract upfront
- Plan UI/UX before writing code
- Manage a multi-week engineering project professionally

WEEKLY ASSESSMENT

Architecture review and scope sign-off session with instructor feedback

Capstone Build Sprint

Execute the planned SaaS capstone: build the backend, frontend, auth, billing, and core features end-to-end following the architecture plan.

LEARNING OBJECTIVES

- Implement the planned database schema and API in FastAPI/Django
- Build the Next.js frontend consuming the API
- Integrate authentication, billing, and core SaaS features
- Apply testing and version control discipline throughout the build

DETAILED TOPIC BREAKDOWN

19.1 Backend Build

- Implementing models, migrations and the database schema
- Building all planned API endpoints with validation
- Implementing authentication and RBAC per the plan
- Integrating Redis caching and Celery background jobs where planned

19.2 Payments & Integrations Build

- Implementing Stripe/Paystack checkout and webhooks
- Implementing file upload features to cloud storage
- Implementing transactional email flows
- Implementing any real-time/WebSocket features planned

19.3 Frontend Build

- Building the Next.js App Router pages/layouts per wireframes
- Implementing forms with Server Actions and client validation
- Building the billing/subscription management UI
- Building the dashboard and core feature UI with Tailwind/shadcn

19.4 Integration & Hardening

- Wiring frontend to backend end-to-end
- Writing tests for critical paths (auth, payments, core feature)
- Fixing bugs found during integration testing
- Performance pass: caching, query optimization, bundle size

19.5 Deployment & Daily Standups

- Setting up CI/CD for the capstone repo
- Deploying staging and production environments
- Daily standup check-ins on milestone progress
- Adjusting scope pragmatically against the timeline

HANDS-ON EXERCISES

- Complete and demo the backend API milestone by mid-week
- Complete and demo the frontend core screens milestone
- Integrate one third-party service (payments or storage) fully
- Fix all critical bugs found in a peer testing session

ASSIGNMENTS

- Build the fully functioning SaaS capstone application end-to-end per the approved architecture
- Deploy a working staging version of the capstone for peer and instructor testing

PROJECTS

- Working End-to-End SaaS Capstone Application

WEEKLY LEARNING OUTCOMES

- Execute a multi-week engineering build against a plan
- Integrate a full stack of technologies into one cohesive product
- Debug and harden a real application under time pressure
- Deliver a deployed, testable product milestone

WEEKLY ASSESSMENT

Mid-sprint and end-of-sprint milestone demos with instructor feedback

Capstone Launch, Review & Career Readiness

Finalize, polish, and present the capstone SaaS product; prepare a professional portfolio and prepare for technical interviews.

LEARNING OBJECTIVES

- Polish and launch the capstone SaaS product
- Present the project professionally to technical and non-technical audiences
- Prepare a portfolio and resume showcasing the work
- Prepare for full-stack technical interviews

DETAILED TOPIC BREAKDOWN

20.1 Final Polish & QA

Full regression testing of all core flows

UI/UX polish pass (empty states, error states, loading states)

Accessibility audit and fixes

Cross-browser and responsive QA

20.2 Production Launch

Final production deployment checklist

Setting up production monitoring/alerts

Domain, SSL and branding finalization

Writing a public-facing README and demo video

20.3 Portfolio & Personal Branding

Writing a case study for the capstone project

Updating GitHub profile and pinning key repositories

Building/updating a personal developer portfolio site

Optimizing a LinkedIn profile for tech recruiters

20.4 Technical Interview Preparation

Common full-stack system design interview questions

Data structures and algorithms review for coding interviews

Behavioral interview preparation (STAR method)

Mock technical interview session

20.5 Final Presentation

Structuring a compelling project demo/pitch

Presenting architecture decisions and tradeoffs

Handling technical Q&A confidently

Peer and instructor final evaluation

HANDS-ON EXERCISES

- Complete a full QA pass and fix all found issues
- Write a portfolio case study for the capstone project
- Complete one mock technical interview with feedback
- Record a 3-5 minute demo video of the finished product

ASSIGNMENTS

- Deliver the final polished, deployed SaaS capstone product with full documentation
- Deliver a live final presentation with architecture walkthrough and Q&A

PROJECTS

- Final Launched SaaS Capstone Product & Portfolio

WEEKLY LEARNING OUTCOMES

- Ship a polished, production-quality SaaS product
- Present technical work confidently to any audience
- Have an interview-ready portfolio and resume
- Be ready for full-stack developer job interviews

WEEKLY ASSESSMENT

Final capstone presentation graded on functionality, code quality, architecture and presentation skills